Database Systems Reference Guide

Ahmed Yassin

October 14, 2026

Contents

I Introduction to Database Systems **5**

1 Core Concepts & Architectures **5**
1.1 What is a Database? . . . 5
1.2 Database vs. DBMS vs. DBS . . . 5
1.3 Data vs. Metadata . . . 5
1.4 Problems in File-Based Systems (FBS) . . . 5
1.5 Types of Databases . . . 6
1.6 Database Life Cycle (DBLC) . . . 6

II Database Design: ERD & EERD **7**

2 The Entity-Relationship Diagram (ERD) **7**
2.1 Introduction . . . 7
2.2 Entities: Strong vs. Weak . . . 7
2.3 Types of Attributes . . . 7
2.4 Degree of Relationships . . . 8
2.5 Cardinality Constraints . . . 8
2.6 Participation Constraints . . . 8
2.7 Database Keys . . . 8
2.8 Company Case Study ERD . . . 9

3 Enhanced Entity-Relationship Diagram (EERD) **11**
3.1 Supertype and Subtype Entities . . . 11
3.2 Completeness Constraints . . . 11
3.3 Disjoint Constraints . . . 11
3.4 Vehicle EERD Case Study . . . 12

III Relational Mapping **13**

4 Mapping ERD to Relational Schema **13**
4.1 Introduction . . . 13
4.2 Step 1: Mapping of Regular (Strong) Entity Types . . . 13
4.3 Step 2: Mapping of Weak Entity Types . . . 13

4.4 Step 3: Mapping of Binary 1:1 Relationship Types . . . 13
4.5 Step 4: Mapping of Binary 1:M Relationship Types . . . 13
4.6 Step 5: Mapping of Binary M:N Relationship Types . . . 14
4.7 Step 6: Mapping of Unary (Recursive) Relationships . . . 14
4.8 Step 7: Mapping of N-ary (Ternary) Relationships . . . 14
4.9 EERD Mapping Strategies . . . 14
4.10 Case Study Mapping Solution . . . 15
4.11 Implementing the Mapping in T-SQL . . . 16

IV Database Normalization 18

5 Normalization Theory & Normal Forms 18
5.1 Database Anomalies . . . 18
5.2 Functional Dependencies (FD) . . . 18
5.3 The Normal Forms . . . 18
5.4 Normalization Example . . . 19
5.4.1 Normalization Solution . . . 19

V Microsoft SQL Server Architecture & T-SQL 20

6 SQL Server Architecture & Installation 20
6.1 System Databases . . . 20
6.2 Database Files . . . 20
6.3 Authentication Modes . . . 20
6.4 Database Instances . . . 20
6.5 Database Integrity Levels . . . 20

7 T-SQL Statement Categories 21
7.1 The SQL Standard & Dialects . . . 21
7.2 T-SQL Categories . . . 21

8 T-SQL Data Types 22
8.1 Numeric Data Types . . . 22
8.2 Character String Data Types . . . 22
8.3 Date and Time Data Types . . . 22
8.4 Special and Deprecated Data Types . . . 22

VI Querying & Data Manipulation 23

9 Fundamentals of SELECT & DML 23
9.1 Basic DDL & DML Commands . . . 23
9.2 DQL Projection and Filtering . . . 24

10 Joins 25
10.1 Types of Joins . . . 25
10.2 Join Demonstrations . . . 25
10.2.1 Join Examples . . . 26
10.2.2 DML Statements using Joins . . . 26

11 Functions, Grouping & Query Mechanics **27**
11.1 Aggregate Functions . . . 27
11.2 GROUP BY and HAVING . . . 27
11.3 Null Handling & Concatenation . . . 27
11.4 Pattern Matching (LIKE operator) . . . 27
11.5 Execution Order in SQL Server . . . 28
11.6 Conditional Logic (CASE & IIF) . . . 28
11.7 Data Type Conversion Functions . . . 29
11.8 Date Operations . . . 29

12 Subqueries & Set Operations **30**
12.1 Subqueries . . . 30
12.2 Set Operations (Union Family) . . . 30

13 Advanced Query Techniques **31**
13.1 TOP and TOP WITH TIES . . . 31
13.2 SELECT INTO (Copying Data) . . . 31
13.3 Random Ordering . . . 31
13.4 Ranking Functions . . . 31
13.5 Offset & Analytic Window Functions . . . 32
13.6 Aggregates: ROLLUP, CUBE, and GROUPING SETS . . . 32
13.7 Pivot and Unpivot . . . 33

VII Constraints & Data Integrity **34**

14 SQL Constraints & Referential Actions **34**
14.1 Supported Constraints . . . 34
14.2 Referential Actions (Foreign Key Options) . . . 34
14.3 DROP vs. DELETE vs. TRUNCATE . . . 35

15 Rules, Defaults & Custom Datatypes **36**
15.1 Rules . . . 36
15.2 Defaults . . . 36
15.3 User-Defined Datatypes . . . 36
15.4 Persisted Computed Columns . . . 37

VIII Database Schemas & Objects **38**

16 Schemas & Synonyms **38**
16.1 Database Schemas . . . 38
16.2 Synonyms . . . 38

IX Programming in T-SQL **39**

17 Variables, Control of Flow & Transactions **39**
17.1 Variables . . . 39
17.2 Table Variables . . . 39
17.3 Dynamic SQL . . . 40

17.4 Control of Flow . . . 40
17.5 Error Handling (TRY-CATCH) . . . 41
17.6 Transactions (ACID Properties) . . . 41

18 User-Defined Functions (UDF) 42
18.1 Scalar Functions . . . 42
18.2 Inline Table-Valued Functions . . . 42
18.3 Multi-Statement Table-Valued Functions . . . 43

19 Views 44
19.1 Introduction . . . 44
19.2 Indexed (Materialized) Views . . . 44
19.3 View Encryption & Check Option . . . 44

20 Stored Procedures 45
20.1 Introduction . . . 45
20.2 Parameters & Return Values . . . 45

21 Triggers 46
21.1 Introduction . . . 46
21.2 Virtual Tables: `inserted` and `deleted` . . . 46
21.3 `AFTER` vs. `INSTEAD OF` Triggers . . . 46

22 The MERGE Statement 47

X Indexes & Performance Optimization 48

23 Database Indexes 48
23.1 Table Scan vs. Index Seek . . . 48
23.2 Clustered vs. Non-Clustered Indexes . . . 48

24 Cursors 49
24.1 Cursor Lifecycle . . . 49

XI Backup, Recovery & Integration 50

25 Backup & Recovery Models 50
25.1 Recovery Models . . . 50
25.2 Backup Categories . . . 50
25.3 Database Snapshots . . . 50
25.4 Identity Gap Resolution & Bulk Insert . . . 51

26 XML Integration 52

27 SQL Common Language Runtime (SQL CLR) 53

A SQL Execution Order Quick Reference 54

PART I

Introduction to Database Systems

1 Core Concepts & Architectures

1.1 What is a Database?

A **Database (DB)** is a structured collection of logically related data designed to meet the information needs of an organization. In a relational database, data is represented in tabular structures consisting of rows (tuples) and columns (attributes).

1.2 Database vs. DBMS vs. DBS

These three terms are often confused:

Database (DB): The actual collection of tables, indexes, and stored records (analogous to a library book).

Database Management System (DBMS): The software suite used to create, maintain, query, secure, and administer databases, such as Microsoft SQL Server, PostgreSQL, MySQL, or Oracle (analogous to a librarian).

Database System (DBS): The complete system that encompasses the DB, the DBMS, the physical hardware, operating systems, networking infrastructure, and the end users/applications (analogous to the entire library system).

1.3 Data vs. Metadata

Data: The raw facts or actual content stored within the database fields (e.g., `"Ahmed", 20, "Cairo"`).

Metadata: "Data about data" that defines the structure, schema, constraints, data types, and relationships of the actual data (e.g., specifying that `Name` is a `VARCHAR(50)` and `Age` is an `INT`).

1.4 Problems in File-Based Systems (FBS)

Before DBMSs, each application kept its data in its own separate files. This caused the following problems:

1. **Data Redundancy:** The same information is stored multiple times in different files, wasting storage space.
2. **Data Inconsistency:** Updates to data in one file might not propagate to other files, leading to conflicting records.
3. **Lack of Data Integrity:** No automated check constraints or validation rules to ensure data correctness.
4. **Limited Data Sharing:** Files are siloed, making multi-user access and sharing difficult.
5. **Poor Security:** Lack of fine-grained access control or role-based user authentication.
6. **Difficult Data Retrieval:** Fetching records requires writing custom processing scripts rather than using a standardized declarative language (like SQL).
7. **No Transaction Support:** Lack of ACID properties, exposing data to corruption during system failures.
8. **Program–Data Dependence:** The file structure is defined inside each program, so changing a file's format means changing every program that uses it.

1.5 Types of Databases

Databases are categorized by their data models and deployment targets:

Database Type	Description	Examples
Relational (RDBMS)	Organized as tables with keys	SQL Server, PostgreSQL, MySQL
NoSQL	Flexible formats (documents, key-value, graphs)	MongoDB, Redis, Neo4j, Cassandra
Distributed	Data partitioned across multiple nodes	Google Spanner, CockroachDB
Object-Oriented	Stores data directly as OOP objects	ObjectDB, db4o
Time-Series	Optimized for time-stamped log data	InfluxDB, TimescaleDB
Cloud	Managed cloud-platform deployments	Amazon RDS, Google Firestore
Embedded	Serverless databases running within applications	SQLite, Berkeley DB

Table 1: Taxonomy of Database Systems

1.6 Database Life Cycle (DBLC)

The development of a database follows a structured lifecycle:

1. **Analysis:** System analysts define user requirements, business rules, and scope.
2. **Database Design (ERD):** Database designers build the conceptual schema using Entity-Relationship Diagrams.
3. **Database Mapping:** Converting the ERD into logical relational tables with primary and foreign keys.
4. **Normalization:** Refining the mapped relations to eliminate redundancy and anomalies (applying 1NF, 2NF, 3NF, BCNF). Normalization operates on relations (tables), so it follows mapping.
5. **Implementation:** Implementing the relational schema in RDBMS (SQL Server, MySQL).
6. **Database Administration:** Managing security, backup, performance tuning, and user access.
7. **Data Science & Analytics:** Extracting, analyzing, and visualizing data for decision-making.

PART II

Database Design: ERD & EERD

2 The Entity-Relationship Diagram (ERD)

2.1 Introduction

An **Entity-Relationship Diagram (ERD)** is a visual conceptual representation of a database schema. It identifies the entities, attributes, and relationships required by a business system using Chen's standard notation.

2.2 Entities: Strong vs. Weak

Strong (Independent) Entity: An entity that exists independently of other entities in the system. It possesses a primary key that uniquely identifies each of its records (e.g., `Doctor` with identifier `DoctorID`). Represented as a **single rectangle**.

Weak Entity: An entity that cannot exist without a related parent (strong) entity. It does not have a primary key of its own; instead, it has a **partial key** (or discriminator) that distinguishes records only within the scope of a parent record. Its full unique identification requires combining the parent's primary key with its own partial key. Weak entities always have total participation in their identifying relationship. Represented as a **double rectangle**, its identifying relationship as a **double diamond**, and its partial key with a **dashed underline** (e.g., DepName).

2.3 Types of Attributes

Attributes describe the properties of entities or relationships:

1. **Simple (Atomic):** Cannot be subdivided further (e.g., `FirstName`, `Salary`).
2. **Composite:** Can be split into smaller sub-components (e.g., `Name` → `Fname, Lname`).
3. **Derived:** Not stored; calculated from other stored attributes when needed (e.g., `Age` calculated from `Birthdate`). Represented as a **dashed oval**.
4. **Multivalued:** Can hold multiple values for a single entity instance (e.g., an employee having multiple `PhoneNumbers`). Represented as a **double oval**.
5. **Key Attribute:** Uniquely identifies an entity instance. Represented as an oval with its name **underlined**. An entity may have more than one key (e.g., both `DNUM` and `Dname` are unique); each one is underlined.
6. **Partial Key:** The discriminator of a weak entity. Represented with a **dashed underline**.
7. **Complex:** A nested combination of composite and multivalued attributes (e.g., `Addresses` being multivalued, where each address is composite: `Street, City, Country`).

2.4 Degree of Relationships

The degree of a relationship refers to the number of participating entities:

Unary (Degree 1): A relationship between an entity and itself (e.g., an `Employee` manages other `Employees`).

Binary (Degree 2): A relationship between two entity types (e.g., `Doctor` treats `Patient`).

Ternary (Degree 3): A relationship among three entity types (e.g., `Doctor` prescribes `Medicine` to a `Patient` at a specific `Date`).

N-ary (Degree N): A relationship among more than three entity types.

2.5 Cardinality Constraints

Cardinality defines the numerical ratio of entity occurrences in a relationship:

One-to-One (1:1): Each record in Entity A is related to at most one record in Entity B, and vice versa (e.g., `Manager` manages `Department`).

One-to-Many (1:M): One record in Entity A is associated with multiple records in Entity B, but each record in Entity B is associated with at most one record in Entity A (e.g., `Department` contains `Employees`). "Exactly one" only holds if B also has total participation.

Many-to-Many (M:N): Records in Entity A can relate to multiple records in Entity B, and vice versa (e.g., `Employee` works on multiple `Projects`).

2.6 Participation Constraints

Participation defines whether all instances of an entity type must participate in a relationship:

Total Participation (Mandatory): Every instance of the entity must be involved in the relationship. Represented by a **double line** connecting the entity to the relationship. (For example, every `Car` must belong to a company). Any weak entity must have total participation.

Partial Participation (Optional): Some instances of the entity may participate, but it is not mandatory. Represented by a **single line**. (For example, not every `Employee` is allocated a company car).

2.7 Database Keys

Keys ensure data integrity and define relations:

Primary Key (PK): A column or group of columns that uniquely identifies a row in a table. It cannot contain NULL values and must be unique.

Unique Key: Enforces uniqueness on a column but, unlike the Primary Key, accepts NULL. In SQL Server a `UNIQUE` constraint allows only **one** NULL (the ANSI standard and most other DBMSs allow many).

Foreign Key (FK): A column that references the primary key (or a unique key) of another table, or of the same table for recursive relationships. It enforces referential integrity and can be NULL unless declared `NOT NULL`.

Super Key: Any set of attributes that uniquely identifies a record (may contain redundant attributes).

Candidate Key: A *minimal* super key (no attribute can be removed without losing uniqueness). One candidate key is chosen as the primary key; the others are called *alternate keys*.

Composite Key: A key made of two or more attributes.

2.8 Company Case Study ERD

Consider a company with employees, departments, projects, and dependents:

- Employees have a name (Fname, Lname), SSN (key), birthdate, gender, and salary. Each employee must work for exactly one department, may be supervised by one supervisor (another employee), and works on one or more projects with a specified number of hours.
- Departments have a unique name, unique number (DNUM), locations (multivalued), and exactly one manager (an employee); the manager's hire date is recorded.
- Projects have a unique name, unique number (PNumber), location, and each project must be controlled by one department.
- Dependents are weak entities identified through their employee; each has a name (partial key), gender, and birthdate.

Relationship	Cardinality	Participation
Works_For	Employee M : 1 Department	Both total (every employee has a department; every department has employees)
Manages	Employee 1 : 1 Department	Employee partial, Department total (every department has a manager)
Controls	Department 1 : M Project	Department partial, Project total
Works_On	Employee M : N Project	Both total; attribute `Hours`
Has	Employee 1 : M Dependent	Employee partial, Dependent total (identifying)
Supervises	Employee 1 : M Employee	Partial on both roles (recursive)

Table 2: Company ERD structural constraints

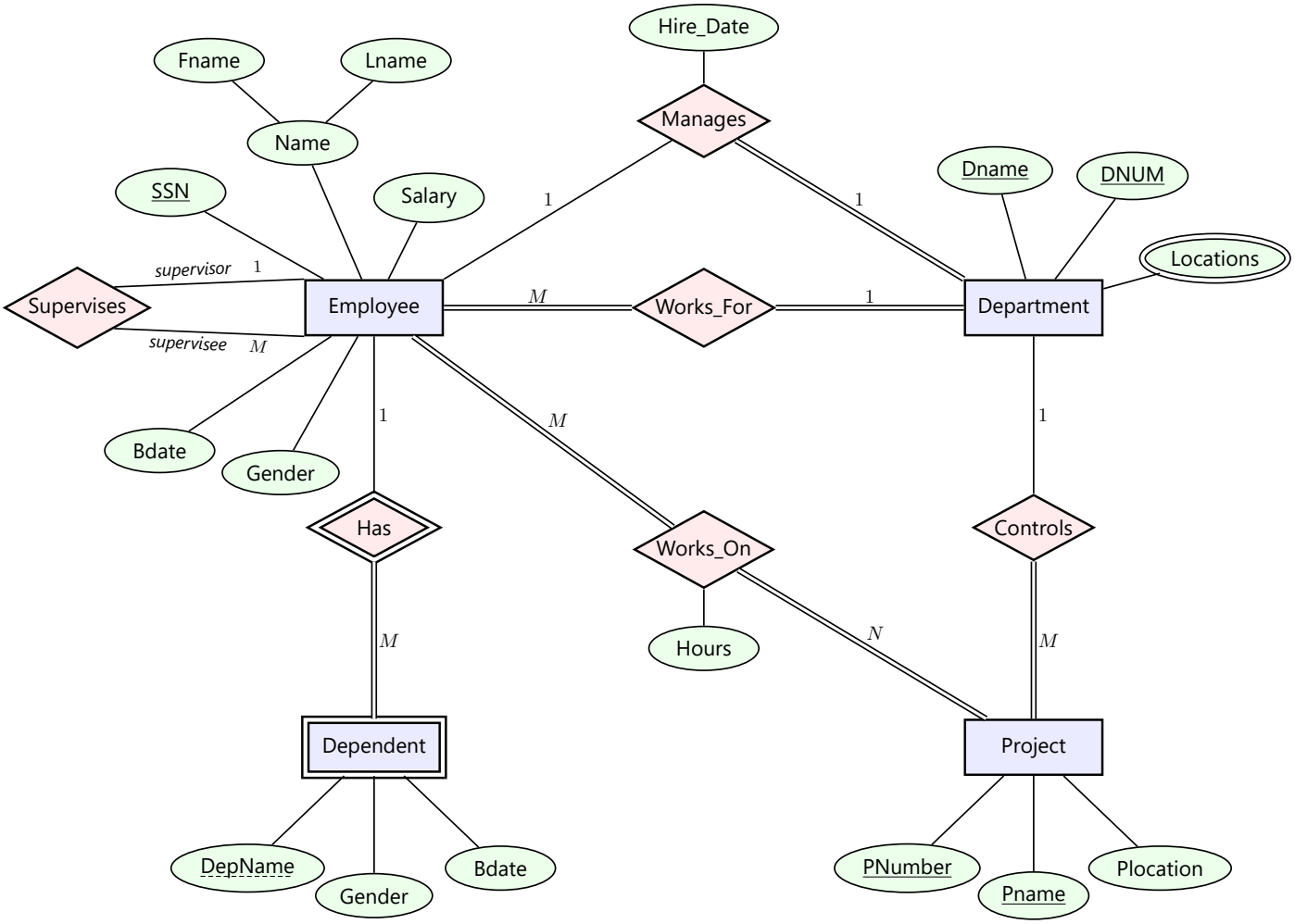


Figure 1: Company ERD (single line = partial, double line = total participation)

3 Enhanced Entity-Relationship Diagram (EERD)

3.1 Supertype and Subtype Entities

Supertype: A generic entity type that has a relationship with one or more subtypes.

Subtype: A subgrouping of entities in an entity type that has attributes distinct from those in other subgroups.

Specialization vs. Generalization: Two directions for building the same hierarchy. *Specialization* is top-down: start from a supertype and define its subtypes (Vehicle $\rightarrow$ Car, Truck). *Generalization* is bottom-up: notice that several entity types share attributes and extract a common supertype (Car, Truck $\rightarrow$ Vehicle). The resulting diagram is the same.

3.2 Completeness Constraints

Total Specialization: Every supertype instance must be a member of a subtype. Represented by a **double line** from the supertype to the specialization circle.

Partial Specialization: A supertype instance does not have to belong to any subtype. Represented by a **single line**.

3.3 Disjoint Constraints

Disjoint Rule (d): An instance can belong to at most one subtype (e.g., a `Vehicle` is either a `Car` or a `Truck`, but not both).

Overlap Rule (o): An instance can belong to multiple subtypes at the same time (e.g., a `Person` can be both a `Student` and an `Employee`).

The two constraints are independent, giving four combinations: total–disjoint, total–overlap, partial–disjoint, and partial–overlap. On each line from the circle to a subtype, a subset symbol ($\subset$, drawn as a U-shaped arc opening toward the circle) indicates that the subtype is a subset of the supertype.

3.4 Vehicle EERD Case Study

A vehicle has a vehicle-id, vehicle-name, and price. It can be specialized as a Car (with make and model) or a Truck (with glass-type, cab-type, and capacity). The specialization is disjoint and total.

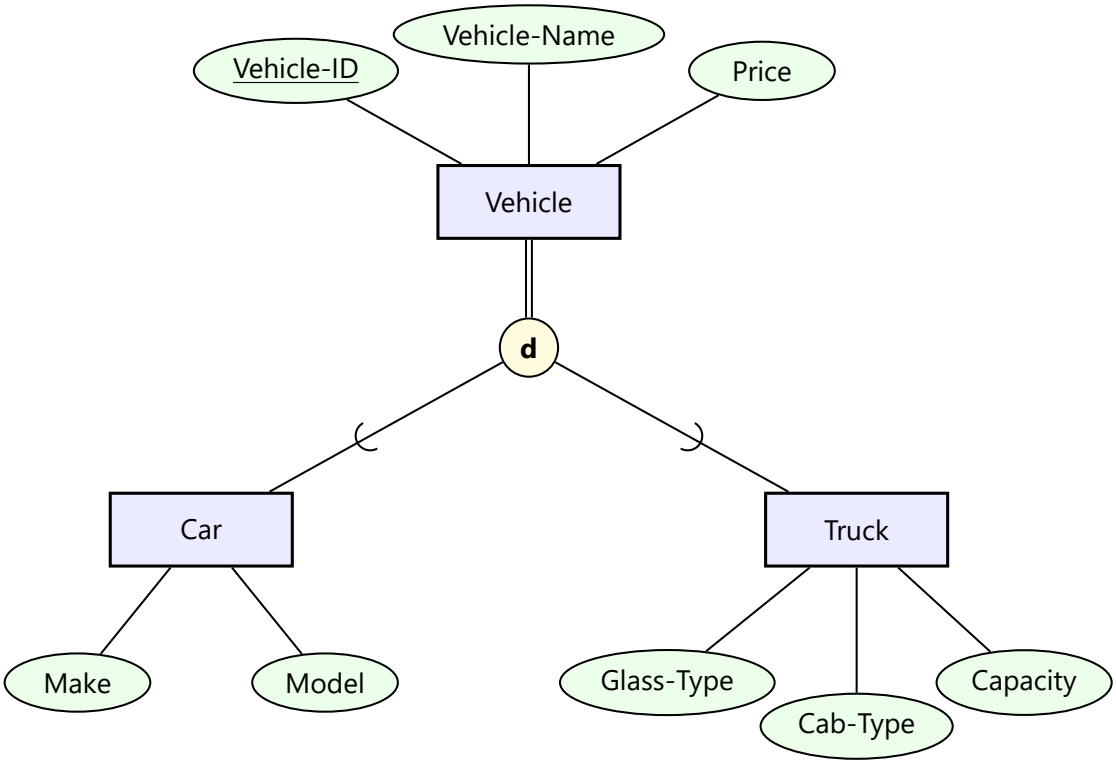


Figure 2: Vehicle Specialization EERD (total, disjoint)

PART III

Relational Mapping

4 Mapping ERD to Relational Schema

4.1 Introduction

Mapping is the process of converting a conceptual model (ERD) into physical database tables.

4.2 Step 1: Mapping of Regular (Strong) Entity Types

For each strong entity type E, create a table T containing all simple attributes of E. The primary key of T is the key attribute of E.

- **Composite Attributes:** Flattened by including only their simple component attributes in the table (e.g., `Name` $\rightarrow$ `Fname`, `Lname` inside the table).
- **Derived Attributes:** Omitted from the physical tables by default. They can be computed dynamically or defined as computed columns.
- **Multivalued Attributes:** Mapped by creating a **new separate table** containing the attribute value and a foreign key referencing the primary key of the parent entity. The primary key of the new table is a composite of the foreign key and the multivalued attribute value.

4.3 Step 2: Mapping of Weak Entity Types

For each weak entity type W with parent entity E, create a table T_W containing the attributes of W. Add the primary key of the table corresponding to E as a foreign key column in T_W. The primary key of T_W is a composite key consisting of the parent's primary key and the weak entity's partial key. Set the foreign key constraint to `ON DELETE CASCADE` to clean up dependents if the parent is deleted.

4.4 Step 3: Mapping of Binary 1:1 Relationship Types

Choose one of the following approaches based on participation:

Foreign Key Approach: Put the primary key of Table A as a foreign key in Table B, and declare it `UNIQUE` so the relationship stays 1:1. If one side has total participation, put the foreign key on that total side to avoid NULL values. Any relationship attributes go in the same table as the foreign key.

Merged Relation Approach: Merge the tables if there is total participation from both sides (1:1 total-total).

4.5 Step 4: Mapping of Binary 1:M Relationship Types

Identify the table representing the "Many" side. Add the primary key of the "One" side table as a foreign key in the "Many" side table. If the relationship has attributes, place them in the "Many" side table as well.

4.6 Step 5: Mapping of Binary M:N Relationship Types

For each M:N relationship, create a **new junction table** R. Include the primary keys of both participating tables as foreign keys in R. The primary key of R is a composite key consisting of both foreign keys. Any attributes associated with the M:N relationship are placed in this new table.

4.7 Step 6: Mapping of Unary (Recursive) Relationships

Unary 1:1: Add a foreign key column in the same table referencing its own primary key, and make it `UNIQUE` (e.g., `SpouseSSN` in `Person`).

Unary 1:M: Add a foreign key column in the same table referencing its own primary key (e.g., `Super_SSN` pointing to `SSN` in the `Employee` table).

Unary M:N: Keep the entity table, and create a separate junction table containing two foreign keys that both reference the entity's primary key (e.g., `Prerequisite(CourseID, PrereqID)`). The primary key of the junction table is the composite of the two foreign keys.

4.8 Step 7: Mapping of N-ary (Ternary) Relationships

For relationships of degree ≥ 3, create a new table. Add the primary keys of all participating entities as foreign keys in this new table, and store any relationship attributes there. For an M:N:P relationship the primary key is the composite of all the foreign keys. If one side has cardinality 1, that side's foreign key is excluded from the primary key, because it is determined by the others.

4.9 EERD Mapping Strategies

When mapping supertypes and subtypes, three main strategies are used:

Strategy 1 (Multiple Tables - Local Attributes): Create a table for the supertype and separate tables for each subtype. Each subtype table contains its local attributes and a primary key that also acts as a foreign key pointing to the supertype table.

Strategy 2 (Single Table - Nullable Columns): Create a single table containing all supertype and subtype attributes. Add a discriminator column (e.g., `VehicleType`) and make all subtype attributes nullable. For overlapping specialization, use one yes/no flag column per subtype instead of a single discriminator.

Strategy 3 (Multiple Tables - Subtypes Only): Create tables for subtypes only, duplicating the supertype attributes in each. Applicable only for **total** specialization (otherwise supertype instances that belong to no subtype would be lost), and best suited to **disjoint** specialization (with overlap, shared data is duplicated across subtype tables).

Applied to the Vehicle EERD:

Strategy 1: **Vehicle**(VehicleID, VehicleName, Price), **Car**(*VehicleID*[FK], Make, Model), **Truck**(*VehicleID*[FK], GlassType, CabType, Capacity)

Strategy 2: **Vehicle**(VehicleID, VehicleName, Price, VehicleType, Make, Model, GlassType, CabType, Capacity)

Strategy 3: **Car**(VehicleID, VehicleName, Price, Make, Model), **Truck**(VehicleID, VehicleName, Price, GlassType, CabType, Capacity)

4.10 Case Study Mapping Solution

The mapped tables for the Company ERD are (primary keys underlined, foreign keys marked FK):

1. **Employee**(SSN, Fname, Lname, Bdate, Gender, Salary, *DNum*[FK], *Super_SSN*[FK])
   (Works_For 1:M → FK on the many side, NOT NULL because participation is total; Supervises → self-referencing FK)
2. **Department**(DNUM, DName (UNIQUE), *MgrSSN*[FK], HireDate)
   (Manages 1:1 → FK on the total side, Department; the relationship attribute goes with it)
3. **Dept_Location**(*DNum*[FK], Location)
4. **Project**(PNumber, PName (UNIQUE), Location, *DNum*[FK])
5. **Works_On**(*ESSN*[FK], *PNum*[FK], Hours)
6. **Dependent**(*ESSN*[FK], DependentName, Gender, Bdate) (FK with `ON DELETE CASCADE`)

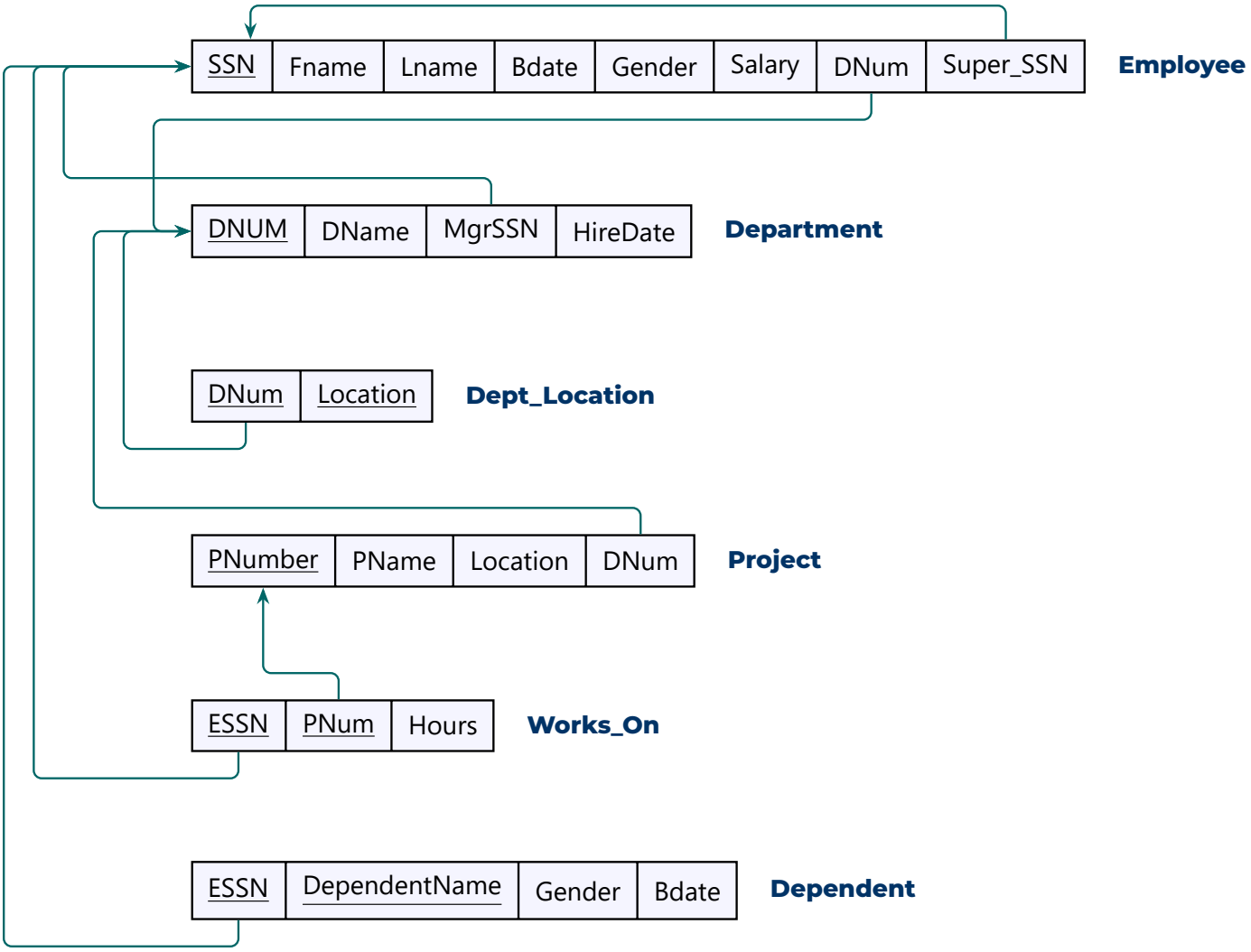


Figure 3: Relational schema of the Company database (underlined = primary key; each arrow runs from a foreign key to the primary key it references)

4.11 Implementing the Mapping in T-SQL

The script below creates the mapped Company schema in its own database. It shows how each ERD constraint becomes a SQL constraint, and where SQL cannot express the ERD directly. `Department` and `Employee` reference each other, so they are created first and linked afterwards:

```
CREATE DATABASE CompanyDB;
GO
USE CompanyDB;
GO

-- Department is created first; its manager FK is added later (circular reference)
CREATE TABLE Department (
    DNUM      INT          PRIMARY KEY,
    DName     VARCHAR(50)  NOT NULL UNIQUE,      -- second key of the ERD
    MgrSSN    CHAR(9)      NULL,                 -- Manages (FK added below)
    HireDate  DATE         NULL                  -- attribute of Manages
);

CREATE TABLE Employee (
    SSN        CHAR(9)       PRIMARY KEY,
    Fname      VARCHAR(30)   NOT NULL,
    Lname      VARCHAR(30)   NOT NULL,
    Bdate      DATE,
    Gender     CHAR(1)       CHECK (Gender IN ('M', 'F')),
    Salary     DECIMAL(10,2) CHECK (Salary > 0),
    DNum       INT           NOT NULL                    -- Works_For: total
       participation
               REFERENCES Department(DNUM),
    Super_SSN  CHAR(9)       NULL                        -- Supervises: recursive,
       partial
               REFERENCES Employee(SSN)
);

-- Close the circle: Department.MgrSSN -> Employee.SSN
ALTER TABLE Department
    ADD CONSTRAINT FK_Department_Manager FOREIGN KEY (MgrSSN) REFERENCES Employee(SSN
       );

-- 1:1 means an employee manages at most one department.
-- A filtered unique index ignores the NULLs that exist while departments are being
   set up.
CREATE UNIQUE INDEX UQ_Department_MgrSSN ON Department(MgrSSN) WHERE MgrSSN IS NOT
   NULL;
```

The remaining tables only reference tables that already exist:

```
-- Multivalued attribute Locations
CREATE TABLE Dept_Location (
    DNum      INT          REFERENCES Department(DNUM) ON DELETE CASCADE,
    Location  VARCHAR(50),
    PRIMARY KEY (DNum, Location)
);
CREATE TABLE Project (
    PNumber   INT          PRIMARY KEY,
    PName     VARCHAR(50)  NOT NULL UNIQUE,
    Location  VARCHAR(50),
    DNum      INT          NOT NULL REFERENCES Department(DNUM)  -- Controls (total)
);
-- M:N relationship with attribute Hours
CREATE TABLE Works_On (
    ESSN   CHAR(9)       REFERENCES Employee(SSN),
    PNum   INT           REFERENCES Project(PNumber),
    Hours  DECIMAL(4,1)  CHECK (Hours >= 0),
    PRIMARY KEY (ESSN, PNum)
);

-- Weak entity: PK = owner's key + partial key, deleted together with its owner
CREATE TABLE Dependent (
    ESSN           CHAR(9)      REFERENCES Employee(SSN) ON DELETE CASCADE,
    DependentName  VARCHAR(30),
    Gender         CHAR(1)      CHECK (Gender IN ('M', 'F')),
    Bdate          DATE,
    PRIMARY KEY (ESSN, DependentName)
);
GO
```

Because `Employee` and `Department` reference each other, rows must be inserted in this order:

```
-- Insertion order breaks the circular dependency:
INSERT INTO Department (DNUM, DName) VALUES (1, 'Research'); -- 1. no manager yet
INSERT INTO Employee VALUES
    ('123456789', 'Ahmed', 'Ali', '1990-03-15', 'M', 12000, 1, NULL); -- 2. employee
UPDATE Department SET MgrSSN = '123456789', HireDate = '2024-01-01'
WHERE DNUM = 1; -- 3. assign manager
```

What Foreign Keys Cannot Enforce

- **Circular total participation:** the ERD says every department *must* have a manager, but `MgrSSN` has to be nullable. SQL Server has no deferred constraints, so the department must exist before its first employee can reference it. Make it `NOT NULL` after loading the data, or enforce it with a trigger.
- **Total participation of the referenced side:** a foreign key can force every employee to have a department, but not every department to have employees. Likewise, "every employee works on at least one project" (M:N) cannot be declared. These rules need triggers or application logic.

PART IV

Database Normalization

5 Normalization Theory & Normal Forms

5.1 Database Anomalies

Normalization is the process of organizing data to reduce redundancy and the anomalies it causes. It is applied to newly mapped tables during design, and also to existing databases that already suffer from the following anomalies:

Insertion Anomaly: The inability to insert certain data without introducing duplicate information or placeholder values (e.g., adding a new course requires creating a dummy student if they are in the same table).

Deletion Anomaly: The unintended loss of data when deleting a record (e.g., deleting a student removes the course information if they are stored in the same row).

Modification (Update) Anomaly: Redundant data updates that must be executed on multiple rows, risking partial updates (e.g., changing a teacher's name requires updating hundreds of student rows).

5.2 Functional Dependencies (FD)

A functional dependency $X \to Y$ states that the value of column X uniquely determines the value of column Y:

1. **Full Functional Dependency:** Attribute Y is fully functionally dependent on a composite primary key X if it depends on the whole of X, and not on any subset of X.
2. **Partial Functional Dependency:** Occurs when a non-key attribute is determined by only a part of a composite primary key.
3. **Transitive Functional Dependency:** Occurs when a non-key attribute Z is determined by another non-key attribute Y, which is in turn determined by the primary key X ($X \to Y \to Z$).

5.3 The Normal Forms

Normalization is applied step-by-step:

First Normal Form (1NF): A table is in 1NF if and only if all attributes contain only atomic (indivisible) values, and there are no repeating groups or multivalued columns.

Second Normal Form (2NF): A table is in 2NF if it is in 1NF and contains **no partial dependencies**. Every non-key attribute must be fully dependent on the entire primary key.

Third Normal Form (3NF): A table is in 3NF if it is in 2NF and contains **no transitive dependencies**. Non-key attributes must not depend on other non-key attributes.

Boyce–Codd Normal Form (BCNF): A stricter 3NF: for every non-trivial dependency $X \to Y$, X must be a super key. It matters only when a table has several overlapping candidate keys.

5.4 Normalization Example

Consider the following student record table. Its values are atomic, so it is already in 1NF, but it still suffers from all three anomalies:

SID (PK)	Sname	Bdate	City	ZipCode	Subject	Grade	Teacher
1	Ahmed	1980-01-01	Cairo	1010	DB	A	Hany
1	Ahmed	1980-01-01	Cairo	1010	Math	B	Eman
2	Ali	1983-05-12	Alex	1111	DB	B	Hany

This table has:

- Composite key: `(SID, Subject)`.
- Full dependency: `(SID, Subject)` $\rightarrow$ `Grade`.
- Partial dependency: `SID` $\rightarrow$ `Sname, Bdate, City, ZipCode`. (Violates 2NF).
- Partial dependency: `Subject` $\rightarrow$ `Teacher` (`Subject` is only part of the key, assuming one teacher per subject). (Violates 2NF).
- Transitive dependency: `SID` $\rightarrow$ `ZipCode` $\rightarrow$ `City`. (Violates 3NF; it appears once the 2NF split puts `ZipCode` and `City` in `Student`).

5.4.1 Normalization Solution

To resolve these anomalies, we split the table into four 3NF relations:

1. **Student**(SID, Sname, Bdate, *ZipCode*^{FK}) (2NF: removes partial dependency on `SID`)
2. **Zip_City**(ZipCode, City) (3NF: removes the transitive dependency)
3. **Course_Teacher**(Subject, Teacher) (2NF: removes partial dependency on `Subject`)
4. **Enrollment**(*SID*^{FK}, *Subject*^{FK}, Grade) (only fully dependent attributes remain)

PART V

Microsoft SQL Server Architecture & T-SQL

6 SQL Server Architecture & Installation

6.1 System Databases

When SQL Server is installed, it initializes four visible system databases (plus the hidden, read-only **Resource** database that stores system objects):

Master: Holds system-level configurations, logins, and metadata for all other databases.

Model: Template database. Any tables, types, or settings added to this database are inherited by any newly created databases.

MSDB: Used by SQL Server Agent for jobs, schedules, and alerts; also stores backup/restore history and Database Mail data.

TempDB: A shared temporary workspace for temporary tables, table variables, and intermediate results of sorts and hash joins. It is re-created every time SQL Server restarts.

6.2 Database Files

SQL Server stores databases in physical files:

.mdf (Primary Data File): Contains the startup information and the actual database data (tables, indexes).

.ldf (Transaction Log File): Stores the sequence of all transactions and log entries for recovery purposes.

.ndf (Secondary Data File): Optional files used to spread data across multiple physical drives.

6.3 Authentication Modes

SQL Server offers two authentication *methods*:

1. **Windows Authentication:** Integrates with Windows credentials (SSPI); no separate password needed.
2. **SQL Server Authentication:** Uses logins and passwords managed directly inside SQL Server.

The server operates in **Windows Authentication mode** (method 1 only) or **Mixed Mode** (both allowed). SQL Server Authentication cannot be used alone.

6.4 Database Instances

An instance is a complete running copy of the SQL Server database engine:

Default Instance: Runs under the computer name (only one per machine).

Named Instance: Identified by computer name + instance name (e.g., `SERVER\SQLEXPRESS`). Multiple named instances can coexist.

6.5 Database Integrity Levels

Integrity constraints ensure database consistency across four levels:

Domain Integrity: Ensures valid values inside a column (data types, default values, check constraints, rules).

Entity Integrity: Ensures each row can be uniquely identified (Primary Key, Unique constraints).

Referential Integrity: Ensures consistency across related tables (Foreign Key constraints).

User-Defined Integrity: Business rules that do not fit the other levels, enforced with triggers, stored procedures, or application code (e.g., "an employee cannot earn more than their manager").

7 T-SQL Statement Categories

7.1 The SQL Standard & Dialects

ANSI SQL is the standard SQL syntax. Different database vendors implement custom extensions (dialects). In Microsoft SQL Server, this dialect is **Transact-SQL (T-SQL)**.

7.2 T-SQL Categories

T-SQL commands are divided into five main categories:

1. **DDL (Data Definition Language):** Defines the database structures and schemas.
   - `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`, `CREATE DATABASE`.
2. **DML (Data Manipulation Language):** Modifies data values.
   - `INSERT`, `UPDATE`, `DELETE`, `MERGE`.
3. **DQL (Data Query Language):** Retrieves data. (Microsoft's documentation counts `SELECT` as part of DML; ITI lists it separately.)
   - `SELECT`.
4. **DCL (Data Control Language):** Manages security permissions.
   - `GRANT`, `REVOKE`.
5. **TCL (Transaction Control Language):** Manages transaction blocks.
   - `BEGIN TRANSACTION`, `COMMIT`, `ROLLBACK`, `SAVE TRANSACTION`.

8 T-SQL Data Types

Choose the smallest type that safely holds the data: smaller rows mean less storage, less memory, and faster queries.

8.1 Numeric Data Types

`BIT`: Holds 0, 1, or NULL.

`TINYINT`: 1 byte. Range: 0 to 255.

`SMALLINT`: 2 bytes. Range: -32,768 to 32,767.

`INT`: 4 bytes. Range: -2,147,483,648 to 2,147,483,647.

`BIGINT`: 8 bytes. Range: -2^{63} to $2^{63}-1$.

`DECIMAL(p, s)` / `NUMERIC(p, s)`: Exact fixed-point number with precision p (total digits) and scale s (digits after the point).

`MONEY` / `SMALLMONEY`: Exact currency values with 4 decimal places.

`FLOAT(n)` / `REAL`: Approximate floating-point numbers. Avoid them for money, because they cannot represent many decimal fractions exactly.

8.2 Character String Data Types

`CHAR(n)`: Fixed-length non-Unicode string (1 byte per character in the column's code page); padded with spaces.

`VARCHAR(n)`: Variable-length non-Unicode string. `VARCHAR(MAX)` stores up to 2 GB.

`NCHAR(n)`: Fixed-length Unicode string (2 bytes per character).

`NVARCHAR(n)`: Variable-length Unicode string. Use it for text that may contain Arabic or other non-Latin characters.

8.3 Date and Time Data Types

`DATE`: Holds date values only (`YYYY-MM-DD`).

`TIME`: Holds time values only (`hh:mm:ss[.nnnnnnn]`).

`DATETIME`: Legacy date and time type; values are rounded to increments of .000, .003, or .007 seconds (about 3.33 ms accuracy).

`DATETIME2`: Modern, high-precision date and time (accurate to 100 nanoseconds).

8.4 Special and Deprecated Data Types

- **Special:** `UNIQUEIDENTIFIER` (stores 16-byte GUIDs generated via `NEWID()`), `XML`, `TABLE`.
- **Deprecated:** `TEXT` (use `VARCHAR(MAX)`), `NTEXT` (use `NVARCHAR(MAX)`), and `IMAGE` (use `VARBINARY(MAX)`).

PART VI

Querying & Data Manipulation

9 Fundamentals of SELECT & DML

Sample Databases Used in This Guide

ITITestDB: Created below. Its `Employee` and `Department` tables (built in *Join Demonstrations*) are the running example for every query from that point to the end of the guide.

CompanyDB: The Company case study, created by the mapping script in the Relational Mapping part.

9.1 Basic DDL & DML Commands

The script below creates a table, changes its structure, and then inserts, updates, and deletes rows:

```
-- Creating a test database
CREATE DATABASE ITITestDB;
GO  -- USE must run in a later batch than CREATE DATABASE
USE ITITestDB;
GO

-- Creating a baseline table
CREATE TABLE Employee (
    ID INT PRIMARY KEY,
    Name VARCHAR(50) DEFAULT 'New Employee',
    Age INT NOT NULL,
    Birthdate DATE
);
-- Altering the table schema
ALTER TABLE Employee ADD Salary INT;
ALTER TABLE Employee ALTER COLUMN Salary FLOAT;
ALTER TABLE Employee DROP COLUMN Salary;

-- Inserting values
INSERT INTO Employee (ID, Name, Age, Birthdate)
VALUES (1, 'Ahmed', 20, '2006-07-27');
-- Inserting multiple values
INSERT INTO Employee VALUES
(2, 'Ali', 25, '2001-05-24'),
(3, 'Omar', 21, '2005-04-12');
-- Updating data
UPDATE Employee
SET Name = 'Ashraf'
WHERE ID = 3;
-- Deleting a row
DELETE FROM Employee
WHERE ID = 2;
```

9.2 DQL Projection and Filtering

```sql
-- Select projection
SELECT ID, Name FROM Employee;

-- Filtering with logical operators
SELECT * FROM Employee WHERE Age > 20;

-- Sorting outputs
SELECT * FROM Employee ORDER BY Age DESC;

-- Distinct output values
SELECT DISTINCT Age FROM Employee;

-- Membership and range queries
SELECT * FROM Employee WHERE Name IN ('Ahmed', 'Ashraf');
SELECT * FROM Employee WHERE Age BETWEEN 20 AND 24;
```

10 Joins

10.1 Types of Joins

Joins combine columns from two or more tables based on matching values:

INNER JOIN: Returns only rows that have a match in both tables.

LEFT JOIN: All rows from the left table, plus matching rows from the right table (NULL where there is no match).

RIGHT JOIN: All rows from the right table, plus matching rows from the left table (NULL where there is no match).

FULL JOIN: All rows from both tables, with NULL if there is no match.

CROSS JOIN: Cartesian product of both tables.

SELF JOIN: Not a separate join type: any of the joins above where a table is joined to itself, using two different aliases.

10.2 Join Demonstrations

The following tables are used by the join examples and by the rest of the guide. The `Employee` table from the previous section is dropped first, because the new one has a different structure:

```sql
DROP TABLE IF EXISTS Employee;   -- SQL Server 2016+
DROP TABLE IF EXISTS Department;

CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);

INSERT INTO Department VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance'),
(4, 'Marketing'),
(5, 'Operations'); -- empty department for outer joins

CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    DepartmentID INT NULL REFERENCES Department(DepartmentID),
    ManagerID INT NULL REFERENCES Employee(EmployeeID)
);

INSERT INTO Employee VALUES
(1, 'Ahmed', 30, 1, NULL), -- HR employee with no manager
(2, 'Sara', 25, 2, 1), -- IT employee managed by Ahmed
(3, 'Ali', 35, 3, 1), -- Finance employee managed by Ahmed
(4, 'Mona', 28, 2, 3), -- IT employee managed by Ali
(5, 'Omar', 40, NULL, NULL), -- Employee with no department and no manager
(6, 'Laila', 29, 4, NULL), -- Marketing employee with no manager
(7, 'Khaled', 38, 3, 2); -- Finance employee managed by Sara
```

10.2.1 Join Examples

```sql
-- 1. Inner Join
SELECT E.Name, D.DepartmentName
FROM Employee E INNER JOIN Department D
ON E.DepartmentID = D.DepartmentID;

-- 2. Left Outer Join
SELECT E.Name, D.DepartmentName
FROM Employee E LEFT OUTER JOIN Department D
ON E.DepartmentID = D.DepartmentID;

-- 3. Right Outer Join
SELECT E.Name, D.DepartmentName
FROM Employee E RIGHT OUTER JOIN Department D
ON E.DepartmentID = D.DepartmentID;

-- 4. Full Outer Join
SELECT E.Name, D.DepartmentName
FROM Employee E FULL OUTER JOIN Department D
ON E.DepartmentID = D.DepartmentID;

-- 5. Cross Join
SELECT E.Name, D.DepartmentName
FROM Employee E CROSS JOIN Department D;

-- 6. Self Join (Hierarchical: Employee and Manager names)
-- E2 represents the parent (manager), while E1 represents the child (employee) which
    contains the foreign key referring to the manager.
SELECT E1.Name AS EmployeeName, E2.Name AS ManagerName
FROM Employee E1 INNER JOIN Employee E2
ON E2.EmployeeID = E1.ManagerID;
```

10.2.2 DML Statements using Joins

In T-SQL, UPDATE and DELETE can use a join to decide which rows to change:

```sql
-- Updating employees based on a join condition
UPDATE E
SET E.Age += 1
FROM Employee E INNER JOIN Department D
ON E.DepartmentID = D.DepartmentID
WHERE D.DepartmentName = 'Finance';

-- Deleting rows based on a join condition
DELETE E
FROM Employee E INNER JOIN Department D
ON E.DepartmentID = D.DepartmentID
WHERE D.DepartmentName = 'Marketing';
```

11 Functions, Grouping & Query Mechanics

11.1 Aggregate Functions

Aggregate functions summarize multiple values into a single output:

- `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`.

11.2 GROUP BY and HAVING

The `GROUP BY` clause groups matching rows together. The `HAVING` clause acts as a filter on these aggregated groups (unlike `WHERE`, which filters individual rows before grouping):

```sql
SELECT DepartmentID, COUNT(*) AS TotalEmployees, SUM(Age) AS TotalAge
FROM Employee
WHERE Age > 20
GROUP BY DepartmentID
HAVING COUNT(*) > 1;
```

11.3 Null Handling & Concatenation

`ISNULL(expr, replacement)`: Replaces NULL with a specified fallback value (takes exactly two arguments).

`COALESCE(val1, val2, ...)`: Returns the first non-NULL expression in the arguments list.

`CONCAT(val1, val2, ...)`: Combines string values, automatically converting numeric inputs and treating NULL values as empty strings.

```sql
-- Replacing NULL manager values with 555
SELECT Name, ISNULL(ManagerID, 555) FROM Employee;

-- First non-NULL value among ManagerID, DepartmentID, and fallback
SELECT Name, COALESCE(ManagerID, DepartmentID, 111) FROM Employee;

-- String concatenation
SELECT CONCAT(Name, ' works in Dept ', DepartmentID) FROM Employee;
```

11.4 Pattern Matching (LIKE operator)

The `LIKE` operator uses `%` (any sequence of zero or more characters), `_` (exactly one character), `[ ]` (one character from a set or range), and `[^]` (one character not in the set). With the default case-insensitive collation, `'a%'` also matches `'Ahmed'`:

```sql
-- This will fetch all values starting with 'a' and ending with 'm'
SELECT * FROM Employee WHERE Name LIKE 'a%m';
```

Pattern	Description	Example Matches
`'a%m'`	Starts with 'a', ends with 'm'	`adam`, `alarm`
`'%g'`	Ends with 'g'	`king`, `running`
`'%v_'`	Second to last character is 'v'	`live`, `cave`
`'[atd]%'`	Starts with 'a', 't', or 'd'	`apple`, `tree`, `dog`
`'[^atd]%'`	Does not start with 'a', 't', or 'd'	`sun`, `moon`
`'[a-r]%'`	Starts with any letter from 'a' to 'r'	`boy`, `pencil`
`'%[%]'`	Ends with the literal character '%'	`50%`

Table 3: LIKE Pattern Matching Wildcards

11.5 Execution Order in SQL Server

Although written in a specific order, SQL Server executes query clauses in a different logical sequence. This *logical* order explains why aliases defined in the `SELECT` clause cannot be referenced in `WHERE`, `GROUP BY`, or `HAVING`. (The optimizer may physically execute the steps differently, but the result is always as if this order were followed.)

1. **`FROM` & `JOIN`:** Defines and joins the source tables.
2. **`WHERE`:** Filters individual rows. It runs before grouping, which is why it cannot contain aggregate functions such as `COUNT(*)`.
3. **`GROUP BY`:** Aggregates rows into groups.
4. **`HAVING`:** Filters the grouped aggregates.
5. **`SELECT`:** Selects columns and computes expressions/aliases.
6. **`DISTINCT`:** Removes duplicate rows.
7. **`ORDER BY`:** Sorts the output rows.
8. **`TOP` / `OFFSET-FETCH`:** Restricts the final row count.

Because `ORDER BY` runs after `SELECT`, it is the only clause that *can* reference a column alias.

11.6 Conditional Logic (CASE & IIF)

`CASE`: The standard conditional logic control structure.

`IIF`: A shorthand equivalent to a ternary operator, useful for simple true/false checks.

```
SELECT Name, Age,
       CASE
           WHEN Age < 25 THEN 'Junior'
           WHEN Age BETWEEN 25 AND 35 THEN 'Mid-Level'
           ELSE 'Senior'
       END AS SeniorityLevel,
       IIF(Age >= 30, 'Experienced', 'Young') AS ExperienceTag
FROM Employee;
```

11.7 Data Type Conversion Functions

The main T-SQL conversion functions are:

`CAST(value AS datatype)`: The standard ANSI SQL type conversion function.

`CONVERT(datatype, value [, style])`: A SQL Server-specific function that includes formatting styles, especially useful when formatting date-to-string values.

`PARSE(string AS datatype)`: Converts a string representation of a value (e.g., currency) to the specified datatype.

`TRY_CAST`, `TRY_CONVERT`, `TRY_PARSE`: Same as their counterparts, but return NULL instead of raising an error when the conversion fails.

```
SELECT CAST('2024-01-15' AS DATE);
SELECT CONVERT(VARCHAR(10), GETDATE(), 103);    -- dd/mm/yyyy
SELECT PARSE('$1,250.50' AS MONEY USING 'en-US');
SELECT TRY_CAST('abc' AS INT);                  -- NULL, no error
```

11.8 Date Operations

```
-- Fetching system times
SELECT GETDATE(), SYSDATETIME();

-- Fetching the end of the month for a specific date
SELECT EOMONTH(GETDATE());

-- Extracting parts of a date
SELECT YEAR(GETDATE()), MONTH(GETDATE()), DAY(GETDATE());
SELECT DATEPART(WEEKDAY, GETDATE()), DATENAME(MONTH, GETDATE());

-- Date arithmetic
SELECT DATEADD(DAY, 30, GETDATE());                      -- 30 days from now
SELECT DATEDIFF(YEAR, '2000-05-20', GETDATE());          -- difference in year
   boundaries
```

12 Subqueries & Set Operations

12.1 Subqueries

A subquery is a nested query enclosed in parentheses. It can be used within search conditions or inside the projection list:

```sql
-- Scalar subquery (returns a single value)
SELECT Name, Age
FROM Employee
WHERE Age > (SELECT AVG(Age) FROM Employee);

-- Subquery in the SELECT list
SELECT Name, (SELECT COUNT(*) FROM Employee) AS TotalEmployees
FROM Employee;

-- Multi-valued subquery with IN
SELECT Name
FROM Employee
WHERE DepartmentID IN (SELECT DepartmentID FROM Department WHERE DepartmentName LIKE
    'I%');
```

12.2 Set Operations (Union Family)

Set operators combine the result sets of multiple queries:

`UNION ALL`: Combines result sets directly. Duplicate records are retained. Fast.

`UNION`: Combines result sets and removes duplicate rows. Slower, because removing duplicates requires an extra sort or hash step. The output order is *not* guaranteed; use `ORDER BY` if order matters.

`INTERSECT`: Returns only rows that are common to both queries.

`EXCEPT`: Returns rows from the first query that are not present in the second query.

```sql
-- All queries must return the same number of columns with compatible types.
-- Employees in IT, or older than 30 (each name listed once)
SELECT Name FROM Employee WHERE DepartmentID = 2
UNION
SELECT Name FROM Employee WHERE Age > 30;

-- Employees who are also managers of someone
SELECT EmployeeID FROM Employee
INTERSECT
SELECT ManagerID FROM Employee;

-- Departments that have no employees
SELECT DepartmentID FROM Department
EXCEPT
SELECT DepartmentID FROM Employee;
```

13 Advanced Query Techniques

13.1 TOP and TOP WITH TIES

TOP(N): Returns the first N rows.

TOP(N) WITH TIES: Used in conjunction with `ORDER BY`. If multiple rows have the same value as the Nth row, all matching rows are returned. This can result in more than N rows being returned.

```
-- Returns the 3 oldest employees, including anyone tied with the 3rd
SELECT TOP 3 WITH TIES *
FROM Employee
ORDER BY Age DESC;
```

13.2 SELECT INTO (Copying Data)

Creates a new table and inserts query results into it. Only column names, data types, nullability, the data, and the `IDENTITY` property are copied; constraints, indexes, and triggers are *not*:

```
-- Copying table structure and data
SELECT * INTO EmployeeArchive FROM Employee;

-- Copying table schema only (without data)
SELECT * INTO EmptyEmployeeCopy FROM Employee WHERE 1 = 0;
```

13.3 Random Ordering

To randomly order rows in T-SQL, you can sort by the `NEWID()` function:

```
-- How it works: NEWID() generates a unique GUID for each row.
-- Sorting by it causes the query engine to sort rows in a random order.
SELECT * FROM Employee ORDER BY NEWID();

-- Fetching a random sample of 5 employees
SELECT TOP 5 * FROM Employee ORDER BY NEWID();
```

13.4 Ranking Functions

Ranking functions compute a rank value for each row within a partition:

ROW_NUMBER(): Assigns a unique sequential integer to each row, starting at 1.

RANK(): Assigns ranks. If there are ties, they receive the same rank, and subsequent rank values are skipped.

DENSE_RANK(): Assigns ranks. Ties receive the same rank, but no rank values are skipped.

NTILE(N): Divides rows into N buckets and returns the bucket number for each row.

```
SELECT Name, Age, DepartmentID,
       ROW_NUMBER() OVER(PARTITION BY DepartmentID ORDER BY Age DESC) AS RowNum,
       RANK() OVER(PARTITION BY DepartmentID ORDER BY Age DESC) AS Rnk,
       DENSE_RANK() OVER(PARTITION BY DepartmentID ORDER BY Age DESC) AS DenseRnk,
       NTILE(3) OVER(ORDER BY Age DESC) AS Bucket
FROM Employee;
```

13.5 Offset & Analytic Window Functions

These window functions allow accessing values from other rows without self-joins:

`LAG(col, offset):` Accesses a column value from a previous row.

`LEAD(col, offset):` Accesses a column value from a subsequent row.

`FIRST_VALUE(col):` Accesses the first value in the partition.

`LAST_VALUE(col):` Accesses the last value in the partition.

```
SELECT Name, Age, DepartmentID,
       LAG(Age, 1) OVER(PARTITION BY DepartmentID ORDER BY Age) AS PrevAge,
       LEAD(Age, 1) OVER(PARTITION BY DepartmentID ORDER BY Age) AS NextAge,
       FIRST_VALUE(Name) OVER(PARTITION BY DepartmentID ORDER BY Age) AS Youngest,
       LAST_VALUE(Name)  OVER(PARTITION BY DepartmentID ORDER BY Age
                              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
                                 FOLLOWING) AS Oldest
FROM Employee;
```

Warning / Common Error

With `ORDER BY`, the default window frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Without an explicit frame, `LAST_VALUE` therefore returns the *current* row's value (or the last of its ties), not the last value of the partition.

13.6 Aggregates: ROLLUP, CUBE, and GROUPING SETS

`ROLLUP:` Generates hierarchical subtotal rows from right to left.

`CUBE:` Generates subtotals for all possible combinations of grouped columns.

`GROUPING SETS:` Explicitly defines the grouping combinations to aggregate.

```
SELECT DepartmentID, ManagerID, SUM(Age) AS TotalAge
FROM Employee
GROUP BY ROLLUP(DepartmentID, ManagerID);
```

13.7 Pivot and Unpivot

PIVOT turns distinct row values into column headers. The pivoted column (DepartmentName) and the aggregated column (Age) are consumed by the operator, so they cannot appear in the outer SELECT; every other source column becomes a grouping column. Note that AVG of an INT column returns an INT (truncated); use AVG(Age * 1.0) for decimals.

```
-- Average age per department, as one row
SELECT 'Average Age' AS Metric, [HR], [IT], [Finance]
FROM (
    SELECT D.DepartmentName, E.Age
    FROM Employee E INNER JOIN Department D ON E.DepartmentID = D.DepartmentID
) AS SourceTable
PIVOT (
    AVG(Age) FOR DepartmentName IN ([HR], [IT], [Finance])
) AS PivotTable;

-- UNPIVOT: turning columns back into rows
SELECT Metric, DepartmentName, AvgAge
FROM (SELECT 'Average Age' AS Metric, 30 AS [HR], 27 AS [IT], 36 AS [Finance]) AS
   Wide
UNPIVOT (
    AvgAge FOR DepartmentName IN ([HR], [IT], [Finance])
) AS UnpivotTable;
```

PART VII

Constraints & Data Integrity

14 SQL Constraints & Referential Actions

14.1 Supported Constraints

SQL Server enforces domain, entity, and referential integrity using constraints:

1. **`NOT NULL`:** Prevents a column from accepting NULL values.
2. **`UNIQUE`:** Ensures all values in a column (or composite group) are unique.
3. **`PRIMARY KEY`:** Combines `NOT NULL` and `UNIQUE`; only one per table.
4. **`FOREIGN KEY`:** Enforces referential integrity.
5. **`CHECK`:** Validates values against a logical boolean condition.
6. **`DEFAULT`:** Sets a fallback value if no value is provided during insertion.

14.2 Referential Actions (Foreign Key Options)

When a row in a parent table is updated or deleted, SQL Server can cascade changes to the child table:

`NO ACTION` (Default): Rejects the delete or update on the parent row if dependent rows exist.

`CASCADE`: Automatically deletes or updates corresponding rows in the child table.

`SET NULL`: Sets the foreign key column in the child rows to NULL (the column must be nullable).

`SET DEFAULT`: Sets the foreign key column in the child rows to its defined default value.

```
CREATE TABLE Doctor (
    DoctorID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

CREATE TABLE Patient (
    PatientID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    DoctorID INT REFERENCES Doctor(DoctorID)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

14.3 DROP vs. DELETE vs. TRUNCATE

Feature	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Removes	Selected rows	All rows	Table + structure
`WHERE` clause	Supported	Not supported	Not supported
Logging	Every deleted row	Page deallocations	Page deallocations
Speed on large tables	Slow	Fast	Fast
Identity reset	No	Resets to seed	N/A (table removed)
Rollback	Inside a transaction	Inside a transaction	Inside a transaction
Fires DML triggers	Yes	No	No
Referenced by an FK	Allowed	Not allowed	Not allowed

Table 4: Comparison of DELETE, TRUNCATE, and DROP

A common exam claim is that `TRUNCATE` cannot be rolled back. In SQL Server this is false: all three statements can be rolled back if they run inside an explicit transaction, and none of them can be undone once committed (in autocommit mode, each statement commits immediately).

15 Rules, Defaults & Custom Datatypes

Deprecated Features

Rules and bound defaults are deprecated and will be removed in a future version of SQL Server. They are covered because they appear in the curriculum; in new designs use `CHECK` and `DEFAULT` constraints instead.

15.1 Rules

A **Rule** is a schema-level object that works like a reusable check constraint. It is defined once and bound to one or more columns or user-defined datatypes. A rule is only checked on new `INSERT`/`UPDATE` operations; existing data is not validated when it is bound.

```
-- Create the rule
CREATE RULE AgeRule AS @Age >= 18;
GO

-- Bind the rule to a column
EXEC sp_bindrule 'AgeRule', 'Employee.Age';
```

15.2 Defaults

A **Default** is a schema-level object that defines a fallback value. It is defined once and bound using system stored procedures.

```
-- Create the default
CREATE DEFAULT DefaultCity AS 'Cairo';
GO

-- Bind the default to a character column (its type must be compatible)
ALTER TABLE Employee ADD City VARCHAR(50);
GO
EXEC sp_bindefault 'DefaultCity', 'Employee.City';
```

15.3 User-Defined Datatypes

You can create custom datatypes and bind rules/defaults to them:

```
-- Creating custom datatype (Egyptian national IDs are exactly 14 digits)
CREATE TYPE EgyptianID FROM CHAR(14) NOT NULL;
GO

-- Binding a rule to the datatype: every column of this type inherits it
CREATE RULE EgyptianIDRule AS @id NOT LIKE '%[^0-9]%' AND LEN(@id) = 14;
GO
EXEC sp_bindrule 'EgyptianIDRule', 'EgyptianID';
```

15.4 Persisted Computed Columns

A computed column is a virtual column calculated from other columns. By default it is calculated each time it is read. Adding `PERSISTED` stores the calculated values on disk and updates them automatically whenever the source columns change; the expression must then be deterministic (e.g., `GETDATE()` is not allowed):

```
CREATE TABLE SalaryCalculation (
    ID INT PRIMARY KEY,
    BaseSalary INT,
    Overtime INT,
    NetSalary AS (ISNULL(BaseSalary, 0) + ISNULL(Overtime, 0)) PERSISTED
);
```

PART VIII

Database Schemas & Objects

16 Schemas & Synonyms

16.1 Database Schemas

A **Schema** is a logical container used to group database objects (tables, views, stored procedures) for organization and security management. The default schema in SQL Server is dbo (Database Owner).

```
-- Create a new schema
CREATE SCHEMA HR;
GO

-- Create a table inside the new schema
CREATE TABLE HR.Department (
    ID INT PRIMARY KEY,
    Name VARCHAR(50)
);
```

16.2 Synonyms

A **Synonym** is an alias for a database object, which helps abstract the underlying schema or physical server location:

```
-- Create a synonym for a table in the HR schema
CREATE SYNONYM HRDept FOR HR.Department;

-- Query using the synonym
SELECT * FROM HRDept;

-- Moving an existing object into another schema
ALTER SCHEMA HR TRANSFER dbo.EmptyEmployeeCopy;  -- the empty copy made with SELECT
   INTO
```

PART IX

Programming in T-SQL

17 Variables, Control of Flow & Transactions

17.1 Variables

T-SQL supports declaring local variables using the `DECLARE` statement:

```
-- Declaring and assigning variables
DECLARE @AgeVar INT;
SET @AgeVar = 30;

-- Assigning values from a query
DECLARE @EmployeeName VARCHAR(50);
SELECT @EmployeeName = Name FROM Employee WHERE EmployeeID = 1;
```

17.2 Table Variables

A table variable stores a temporary result set that exists only for the current batch. Like temporary tables, it is physically stored in TempDB (a common misconception is that it lives only in memory). Unlike a temporary table, its contents are *not* undone by `ROLLBACK`:

```
DECLARE @TempEmployeeTable TABLE (
    EmpID INT,
    EmpName VARCHAR(50)
);

INSERT INTO @TempEmployeeTable
SELECT EmployeeID, Name FROM Employee;

SELECT * FROM @TempEmployeeTable;
```

17.3 Dynamic SQL

Dynamic SQL builds a statement as a string and then executes it:

```
DECLARE @Query NVARCHAR(MAX);
DECLARE @TableName NVARCHAR(50) = N'Employee';

SET @Query = N'SELECT * FROM ' + QUOTENAME(@TableName);   -- QUOTENAME guards object
   names
EXEC sp_executesql @Query;

-- Values should be passed as parameters, never concatenated (prevents SQL injection)
EXEC sp_executesql
     N'SELECT * FROM Employee WHERE Age > @MinAge',
     N'@MinAge INT',
     @MinAge = 30;
```

17.4 Control of Flow

T-SQL supports standard control-of-flow statements:

```
-- IF-ELSE and EXISTS check
IF EXISTS (SELECT * FROM Employee WHERE Age > 40)
BEGIN
    PRINT 'Senior employees found.';
END
ELSE
BEGIN
    PRINT 'No senior employees found.';
END;

-- WHILE loop
DECLARE @Counter INT = 1;
WHILE @Counter <= 5
BEGIN
    PRINT @Counter;
    SET @Counter += 1;
END;
```

17.5 Error Handling (TRY-CATCH)

```
BEGIN TRY
    -- Division by zero error
    SELECT 1 / 0;
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS ErrorNum,
        ERROR_MESSAGE() AS ErrorMsg;
END CATCH;
```

`TRY...CATCH` does not catch compile errors (such as a syntax error or a missing table name) in the same batch; these stop the batch before it runs. Use `THROW` inside `CATCH` to re-raise the original error to the caller.

17.6 Transactions (ACID Properties)

A transaction guarantees the ACID properties:

- **Atomicity:** All operations succeed, or all are rolled back.
- **Consistency:** A transaction takes the database from one valid state to another; every constraint holds before and after it.
- **Isolation:** Uncommitted changes are hidden from other concurrent transactions (to the degree set by the isolation level).
- **Durability:** Once committed, changes survive a crash, because they are first written to the transaction log.

```
BEGIN TRANSACTION;
BEGIN TRY
    UPDATE Employee SET Age = 31 WHERE EmployeeID = 1;
    -- Commit transaction if no error occurs
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Roll back only if a transaction is still open
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    PRINT 'Transaction rolled back: ' + ERROR_MESSAGE();
END CATCH;
```

18 User-Defined Functions (UDF)

18.1 Scalar Functions

Returns a single scalar value. When used in a query, it runs once per row, which can be slow on large tables (SQL Server 2019+ can inline simple scalar functions automatically).

```
CREATE FUNCTION dbo.GetSalaryWithTax (@Salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Salary * 1.14;
END;
GO

-- Using scalar function
SELECT dbo.GetSalaryWithTax(10000);
```

18.2 Inline Table-Valued Functions

Returns a table from a single SELECT statement. The optimizer expands it into the calling query like a parameterized view, so it usually performs well.

```
CREATE FUNCTION dbo.GetEmployeesByDept (@DeptID INT)
RETURNS TABLE
AS
RETURN (
    SELECT EmployeeID, Name, Age
    FROM Employee
    WHERE DepartmentID = @DeptID
);
GO

-- A table-valued function is used in FROM, like a table
SELECT * FROM dbo.GetEmployeesByDept(2);
```

18.3 Multi-Statement Table-Valued Functions

Returns a table, allowing multiple procedural statements to populate it. It is slower than inline table-valued functions.

```
CREATE FUNCTION dbo.GetEmployeesWithBonus()
RETURNS @ResultTable TABLE (
    EmpID INT,
    Name VARCHAR(50),
    Bonus DECIMAL(10,2)
)
AS
BEGIN
    INSERT INTO @ResultTable
    SELECT EmployeeID, Name, Age * 100.0 FROM Employee;
    RETURN;
END;
GO

SELECT * FROM dbo.GetEmployeesWithBonus();
```

19 Views

19.1 Introduction

A **View** is a virtual table defined by a query. It stores no data itself; its query runs against the base tables every time the view is used.

19.2 Indexed (Materialized) Views

By creating a unique clustered index on a view, the view's result set is materialized and stored on disk, which speeds up expensive joins and aggregations. The view must be created `WITH SCHEMABINDING`, must reference tables by two-part names (e.g., `dbo.Employee`), and cannot use outer joins, `TOP`, or `DISTINCT`.

19.3 View Encryption & Check Option

`WITH ENCRYPTION`: Obfuscates the view's definition text in the system catalog to prevent users from reading it.

`WITH CHECK OPTION`: Prevents DML modifications through the view that would violate the view's filter condition.

```
CREATE VIEW ITDepartmentEmployees
WITH ENCRYPTION                 -- definition hidden from sp_helptext
AS
SELECT EmployeeID, Name, DepartmentID
FROM Employee
WHERE DepartmentID = 2
WITH CHECK OPTION;              -- rows must stay visible through the view
GO

UPDATE ITDepartmentEmployees SET Name = 'Sara M.' WHERE EmployeeID = 2;  -- succeeds
UPDATE ITDepartmentEmployees SET DepartmentID = 3 WHERE EmployeeID = 2;  -- fails:
   row would leave the view
```

20 Stored Procedures

20.1 Introduction

A **Stored Procedure** is a named, stored collection of T-SQL statements. Its execution plan is compiled on first execution and cached for reuse. It encapsulates business logic, reduces network traffic and parsing overhead, and enhances security by granting permissions on the procedure instead of the underlying tables.

20.2 Parameters & Return Values

Stored procedures support input parameters, output parameters, and an integer return status. By convention, 0 means success; use OUTPUT parameters or result sets, not the return status, to return data:

```
CREATE PROCEDURE GetEmployeeAge
    @EmpID INT,
    @Age   INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;    -- suppress "(n rows affected)" messages
    IF NOT EXISTS (SELECT * FROM Employee WHERE EmployeeID = @EmpID)
        RETURN 1;      -- status code: employee not found

    SELECT @Age = Age FROM Employee WHERE EmployeeID = @EmpID;
    RETURN 0;          -- status code: success
END;
GO

-- Calling the procedure
DECLARE @EmpAge INT, @Status INT;
EXEC @Status = GetEmployeeAge @EmpID = 1, @Age = @EmpAge OUTPUT;
PRINT CONCAT('Status: ', @Status, ', Age: ', @EmpAge);
```

21 Triggers

21.1 Introduction

A **Trigger** is a special stored procedure that runs automatically in response to an event. **DML triggers** fire on `INSERT`, `UPDATE`, or `DELETE` on a table or view; SQL Server also has **DDL triggers** (e.g., on `CREATE TABLE`) and **logon triggers**.

21.2 Virtual Tables: `inserted` and `deleted`

Triggers access two virtual (pseudo) tables that have the same structure as the target table:

- `inserted`: Stores copies of the newly added or updated rows.
- `deleted`: Stores copies of deleted or old pre-update rows.

21.3 `AFTER` vs. `INSTEAD OF` Triggers

`AFTER` Trigger (Default): Runs after the triggering statement succeeds, inside the same transaction, so it can still roll the change back. `FOR` is a synonym for `AFTER`. Allowed on tables only.

`INSTEAD OF` Trigger: Runs in place of the triggering statement, allowing custom validation or writes to the underlying tables. Allowed on tables and views (commonly used to make a multi-table view updatable).

Warning / Common Error

A trigger fires **once per statement**, not once per row. An `UPDATE` that changes 100 rows fires the trigger once, with 100 rows in `inserted` and `deleted`. Always write trigger code as a set-based query over these tables (as below), never as `SELECT @var = ... FROM inserted`.

Note:

Bulk operations such as `BULK INSERT` do not fire triggers unless the `FIRE_TRIGGERS` option is specified.

```
-- Audit Trigger Example
CREATE TABLE EmployeeAuditLog (
    LogID INT IDENTITY PRIMARY KEY,
    Action VARCHAR(10),
    EmpID INT,
    OldName VARCHAR(50),
    NewName VARCHAR(50),
    ChangeTime DATETIME DEFAULT GETDATE()
);
GO

CREATE TRIGGER trg_EmployeeUpdate
ON Employee
AFTER UPDATE
AS
BEGIN
    INSERT INTO EmployeeAuditLog (Action, EmpID, OldName, NewName)
    SELECT 'UPDATE', i.EmployeeID, d.Name, i.Name
    FROM inserted i INNER JOIN deleted d ON i.EmployeeID = d.EmployeeID;
END;
GO
```

22 The MERGE Statement

The `MERGE` statement combines `INSERT`, `UPDATE`, and `DELETE` operations into a single statement, matching records between a target and a source table:

```
-- Merging source updates into target
MERGE TargetTable AS T
USING SourceTable AS S
ON T.ID = S.ID
WHEN MATCHED THEN
    UPDATE SET T.Name = S.Name
WHEN NOT MATCHED BY TARGET THEN               -- in source only
    INSERT (ID, Name) VALUES (S.ID, S.Name)
WHEN NOT MATCHED BY SOURCE THEN               -- in target only
    DELETE;                                    -- MERGE must end with a semicolon
```

PART X

Indexes & Performance Optimization

23 Database Indexes

23.1 Table Scan vs. Index Seek

Table Scan: The database engine scans every page of a table to locate rows (occurs on tables without a clustered index, known as heap tables). Slower.

Index Scan: The database engine scans the entire index structure.

Index Seek: The engine uses the index's B+ Tree search path to jump directly to target rows. Fast.

23.2 Clustered vs. Non-Clustered Indexes

Clustered Index: Determines the logical storage order of rows in the table (the leaf level of the index *is* the data). Because rows can only be sorted in one order, there can be only one clustered index per table. By default, creating a primary key creates a clustered index on its columns, unless a clustered index already exists or `NONCLUSTERED` is specified.

Non-Clustered Index: A separate B+ tree whose leaf level stores the key values plus a row locator: the clustered index key if the table has one, otherwise a physical row ID (RID) on the heap. A table can have up to 999 non-clustered indexes. Creating a unique constraint automatically creates a non-clustered index.

```sql
-- Clustered index (only one per table)
CREATE CLUSTERED INDEX IX_Emp_ID ON EmployeeArchive(EmployeeID);

-- Non-clustered index with included columns (covers the query below)
CREATE NONCLUSTERED INDEX IX_Emp_Dept
ON Employee(DepartmentID) INCLUDE (Name, Age);

-- Can be answered with an index seek (on a 7-row table the optimizer
-- may still prefer a scan, because it is cheaper at that size)
SELECT Name, Age FROM Employee WHERE DepartmentID = 2;

DROP INDEX IX_Emp_Dept ON Employee;
```

24 Cursors

24.1 Cursor Lifecycle

Cursors process rows in a result set sequentially (row-by-row), as an alternative to set-based operations. They should be used sparingly due to high memory and disk I/O overhead. The cursor lifecycle consists of:

1. **DECLARE:** Defines the query.
2. **OPEN:** Executes the query and populates the cursor.
3. **FETCH:** Retrieves rows one by one, checking the fetch status using @@FETCH_STATUS.
4. **CLOSE:** Releases the current result set.
5. **DEALLOCATE:** Removes the cursor definition from memory.

```
-- Fetching names row-by-row
DECLARE @EmpName VARCHAR(50);
DECLARE EmpCursor CURSOR FOR
SELECT Name FROM Employee WHERE Age > 20;

OPEN EmpCursor;
FETCH NEXT FROM EmpCursor INTO @EmpName;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @EmpName;
    FETCH NEXT FROM EmpCursor INTO @EmpName;
END;

CLOSE EmpCursor;
DEALLOCATE EmpCursor;
```

PART XI

Backup, Recovery & Integration

25 Backup & Recovery Models

25.1 Recovery Models

The recovery model controls how the transaction log is maintained, and therefore which backups and restores are possible:

Simple: The log is truncated automatically at checkpoints. No log backups; you can only restore to the last full/differential backup.

Full: Every transaction is fully logged and kept until a log backup. Supports point-in-time recovery.

Bulk-Logged: Like Full, but bulk operations are minimally logged to save space. Point-in-time recovery is not possible within a log backup that contains bulk operations.

```
ALTER DATABASE ITITestDB SET RECOVERY FULL;
```

25.2 Backup Categories

SQL Server supports three main backup types:

Full Backup: Backs up the entire database.

Differential Backup: Backs up only the data changed since the last full backup. Differentials are cumulative, so a restore needs only the full backup plus the latest differential.

Transaction Log Backup: Backs up the log records written since the previous log backup, then allows that part of the log to be reused. Restoring the chain of log backups enables point-in-time recovery.

```
BACKUP DATABASE ITITestDB TO DISK = 'C:\Backups\ITITestDB_Full.bak';
BACKUP DATABASE ITITestDB TO DISK = 'C:\Backups\ITITestDB_Diff.bak' WITH DIFFERENTIAL
    ;
BACKUP LOG      ITITestDB TO DISK = 'C:\Backups\ITITestDB_Log.trn';  -- Full/Bulk-
    Logged only
```

25.3 Database Snapshots

A database snapshot is a read-only, point-in-time view of a database. It uses copy-on-write: a page is copied into the snapshot file only when it is first changed in the source. A snapshot is *not* a backup, because it depends on the source database files. (Available in all editions since SQL Server 2016 SP1.) `NAME` is the logical name of the source data file:

```
CREATE DATABASE ITIDB_Snapshot ON
( NAME = ITITestDB, FILENAME = 'C:\Backups\ITIDB_Snapshot.ss' )
AS SNAPSHOT OF ITITestDB;
```

25.4 Identity Gap Resolution & Bulk Insert

- **Identity Gap:** If SQL Server crashes or restarts, cached identity values can be lost, causing gaps (rolled-back inserts also consume values). Skipped values can be filled manually with `IDENTITY_INSERT`, and since SQL Server 2017 the cache can be disabled with `ALTER DATABASE SCOPED CONFIGURATION SET IDENTITY_CACHE = OFF`.
- **Bulk Insert:** Imports large text/CSV files much faster than row-by-row `INSERT` statements.

```sql
SET IDENTITY_INSERT EmployeeAuditLog ON;
INSERT INTO EmployeeAuditLog (LogID, Action, EmpID) VALUES (5, 'MANUAL', 1);
SET IDENTITY_INSERT EmployeeAuditLog OFF;
```

```sql
-- The file path is read by the SQL Server service, on the server machine
BULK INSERT Employee
FROM 'C:\data\employees.csv'
WITH (
    FIRSTROW = 2,            -- skip the header row
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '\n'
);
```

26 XML Integration

SQL Server supports exporting query results as XML and parsing XML documents:

```
-- Exporting as XML
SELECT EmployeeID, Name FROM Employee FOR XML PATH('Employee');

-- Parsing XML using OPENXML
DECLARE @hDoc INT;
DECLARE @XML XML = '<Employees><Emp ID="10"><Name>Ali</Name></Emp></Employees>';

EXEC sp_xml_preparedocument @hDoc OUTPUT, @XML;

SELECT * FROM OPENXML(@hDoc, '/Employees/Emp')
WITH (ID INT '@ID', Name VARCHAR(50) 'Name');

EXEC sp_xml_removedocument @hDoc;   -- always release the document handle

-- Modern alternative: XML type methods (no handle to manage)
SELECT E.x.value('@ID', 'INT')            AS ID,
       E.x.value('(Name)[1]', 'VARCHAR(50)') AS Name
FROM @XML.nodes('/Employees/Emp') AS E(x);
```

27 SQL Common Language Runtime (SQL CLR)

SQL CLR allows writing stored procedures and user-defined functions in C# or VB.NET and running them inside SQL Server:

```
-- 1. Enable SQL CLR
EXEC sp_configure 'clr enabled', 1;
RECONFIGURE;
GO

-- 2. Register C# Assembly
-- CREATE ASSEMBLY MyAssembly FROM 'C:\Assemblies\MyCode.dll' WITH PERMISSION_SET =
    SAFE;
```

Since SQL Server 2017, `clr strict security` is on by default: every assembly, even `SAFE`, must be signed with a trusted certificate or added with `sys.sp_add_trusted_assembly`.

A SQL Execution Order Quick Reference

Step	Clause	Purpose
1	`FROM` & `JOIN`	Select source tables and merge datasets
2	`WHERE`	Apply row-level filters
3	`GROUP BY`	Aggregate records into groups
4	`HAVING`	Filter aggregated groups
5	`SELECT`	Project final columns and execute expressions
6	`DISTINCT`	Remove duplicate records
7	`ORDER BY`	Sort the final result set
8	`TOP / OFFSET`	Restrict the number of returned records

Table 5: Logical SQL execution order